Incremental Structure-from-Motion with Adaptive Optimization: A Virtual Tour Implementation

Muhammad Hussain Habib
Student ID: 27100016
27100016@lums.edu.pk

Ayaan Ahmed
Student ID: 27100155
27100155@lums.edu.pk

December 9, 2025

Abstract

We present a complete Structure-from-Motion (SfM) pipeline for 3D scene reconstruction and interactive virtual tour generation. Our system processes sequential images to recover camera poses and sparse 3D geometry through incremental registration using Perspective-n-Point (PnP) estimation, followed by iterative Bundle Adjustment for global consistency. The pipeline addresses key challenges including matching complexity through a hybrid temporal-random camera selection strategy, robust outlier removal via statistical pruning, and memory-efficient optimization through sparse Jacobian formulation. We validate our approach on a 332-frame dataset, producing a point cloud of over 300,000 3D points. The reconstruction is integrated into an interactive WebGL-based virtual tour using Three.js, enabling smooth navigation between camera viewpoints through interpolated transitions. Full implementation and demonstration video available at: `https://github.com/Atlantis004/Computer_Vision_Project`

1 Introduction

Structure-from-Motion (SfM) reconstructs three-dimensional scene geometry and camera trajectories from unordered image collections [2], forming the foundation of modern photogrammetry applications ranging from cultural heritage preservation to autonomous navigation. While commercial solutions like Matterport and Reality Capture provide robust reconstruction capabilities, understanding the underlying geometric principles and implementation challenges remains crucial for computer vision practitioners.

This work implements a complete incremental SfM pipeline that transforms sequential video frames into navigable 3D environments. Unlike batch reconstruction methods that process all images simultaneously, our incremental approach mirrors online SLAM systems by registering cameras sequentially through 2D-3D correspondences.

Key Contributions:

- A hybrid temporal-random matching strategy that balances computational efficiency with global consistency by matching against recent cameras plus randomly sampled older ones
- Statistical outlier pruning based on median distance to remove spurious 3D points from triangulation errors
- Sparse Bundle Adjustment formulation using explicit Jacobian sparsity patterns for memory-efficient global optimization
- Complete pipeline integration into an interactive Three.js virtual tour with smooth camera interpolation

The remainder of this report is organized as follows: Section 2 details the mathematical formulation of each pipeline stage, Section 3 presents experimental validation, Section 4 analyzes key implementation challenges and solutions, Section 5 describes the virtual tour architecture, and Section 6 concludes with lessons learned.

2 Methodology

Figure 1 presents an overview of our incremental Structure-from-Motion pipeline, showing the flow from input images through feature extraction, camera registration, and Bundle Adjustment to produce the final 3D point cloud.

2.1 Feature Extraction and Matching

The pipeline begins by detecting Scale-Invariant Feature Transform (SIFT) keypoints [1] in each frame. SIFT provides robustness to scale, rotation, and illumination changes through its multi-scale representation and gradient-based descriptors. For each image I_i, we extract up to 70,000 keypoints $\{\mathbf{p}_j\}$ with corresponding 128-dimensional descriptors $\{\mathbf{d}_j\}$.

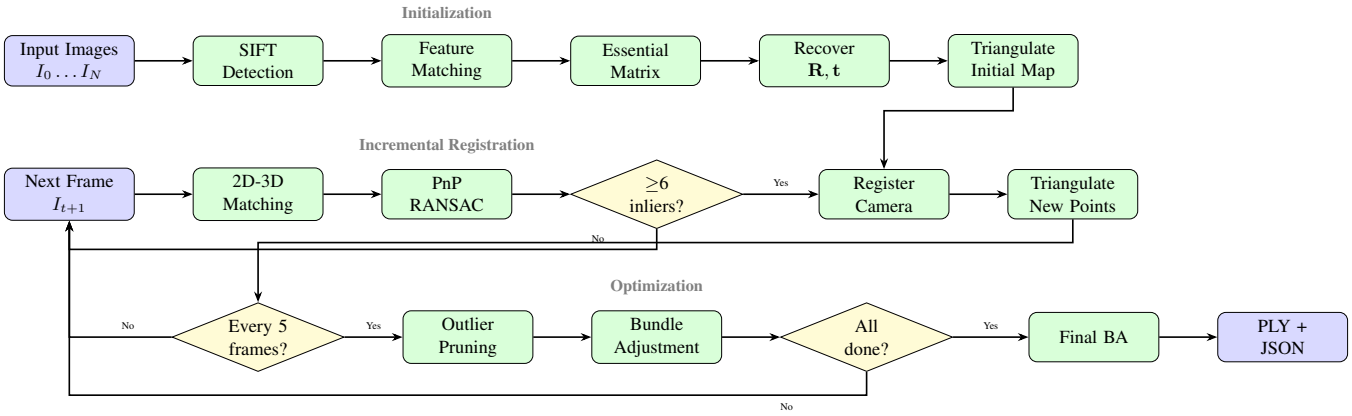

Figure 1: Incremental SfM pipeline. Images are processed through SIFT feature extraction and two-view initialization (top row). New frames are registered via PnP with periodic Bundle Adjustment for drift correction (middle and bottom rows).

Feature matching between frames I_i and I_k employs a brute-force matcher with $k = 2$ nearest neighbors, followed by Lowe's ratio test [1] to reject ambiguous matches:

$$\frac{\|\mathbf{d}_i - \mathbf{d}_{k,1}\|}{\|\mathbf{d}_i - \mathbf{d}_{k,2}\|} < \tau_{\text{ratio}} \tag{1}$$

where $\mathbf{d}_{k,1}$ and $\mathbf{d}_{k,2}$ are the first and second nearest neighbors, and $\tau_{\text{ratio}} = 0.75$. This threshold filters approximately 70% of initial correspondences, retaining only distinctive matches.

To address the $O(N^2)$ complexity of all-pairs matching in sequential data, we implement a hybrid temporal-random matching strategy. When establishing 2D-3D correspondences for PnP, each new frame is matched against the 5 most recent cameras (exploiting temporal coherence) plus 3 randomly sampled older cameras (maintaining global consistency). This balances computational efficiency with robustness to drift.

2.2 Camera Intrinsic Calibration

Accurate camera intrinsics are critical for metric reconstruction. Rather than assuming generic focal lengths, we extract the intrinsic matrix $\mathbf{K}$ from image EXIF metadata using ExifTool. The focal length in pixels is computed from the EXIF focal length (in mm) and sensor dimensions:

$$f_x = f_y = \frac{\max(w, h) \cdot f_{\text{mm}}}{W_{\text{sensor}}} \tag{2}$$

where w and h are image dimensions, f_{mm} is the focal length from EXIF, and W_{sensor} is the sensor width in mm. The principal point (c_x, c_y) is set to the image center. This yields the intrinsic matrix:

$$\mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \tag{3}$$

2.3 Two-View Initialization

Map initialization selects two frames with sufficient baseline to establish metric scale. Given matched normalized coordinates $\{\hat{\mathbf{p}}_i\}$ and $\{\hat{\mathbf{p}}'_i\}$ where $\hat{\mathbf{p}} = \mathbf{K}^{-1}\mathbf{p}$, we estimate the Essential matrix $\mathbf{E}$ using RANSAC:

$$\hat{\mathbf{p}}'^T \mathbf{E} \hat{\mathbf{p}} = 0 \tag{4}$$

The Essential matrix encodes the relative geometry between views and decomposes into rotation $\mathbf{R}$ and translation $\mathbf{t}$ (up to scale):

$$\mathbf{E} = [\mathbf{t}]_\times \mathbf{R} \tag{5}$$

where $[\mathbf{t}]_\times$ is the skew-symmetric matrix of $\mathbf{t}$.

Decomposing $\mathbf{E}$ yields four candidate solutions. We perform cheirality checking by triangulating points under each hypothesis and selecting the pose $[\mathbf{R}|\mathbf{t}]$ that places the maximum number of points in front of both cameras (positive depth $Z > 0$).

Initial triangulation uses the Direct Linear Transform (DLT) on projection matrices $\mathbf{P}_1 = \mathbf{K}[\mathbf{I}|\mathbf{0}]$ and $\mathbf{P}_2 = \mathbf{K}[\mathbf{R}|\mathbf{t}]$ to recover homogeneous world coordinates $\tilde{\mathbf{X}} = [X, Y, Z, W]^T$, which are normalized to Euclidean form $\mathbf{X} = [X/W, Y/W, Z/W]^T$.

2.4 Incremental Camera Registration

Once initialized, subsequent frames are registered through Perspective-n-Point (PnP) estimation [4]. For each candidate frame, we establish 2D-3D correspondences by matching its keypoints against multiple previous cameras that already have associated 3D map points. To avoid matching the same keypoint to multiple 3D points, we maintain a lookup table that maps each new keypoint to at most one 3D point.

We employ a RANSAC-based PnP solver with a reprojection threshold of $\epsilon_{\text{reproj}} = 20.0$ pixels, 1000 iterations, and 99%

confidence. The relatively generous threshold accommodates our high-resolution images where pixel distances are larger. The algorithm requires a minimum of 6 correspondences to attempt pose estimation.

After successful registration, we triangulate new 3D points by matching the current frame against the previous camera. We filter matches to exclude keypoints that are already associated with existing 3D points, ensuring we only triangulate truly new geometry. A cheirality check verifies that triangulated points have positive depth (lie in front of both cameras) before adding them to the map.

2.5 Bundle Adjustment

As we add more cameras, small errors accumulate and cause drift in the reconstruction. Bundle Adjustment (BA) [3] corrects this by refining all camera poses and 3D points together to minimize reprojection error.

Running BA on many cameras and thousands of points would normally exhaust memory. However, SfM problems are naturally sparse as each measurement only involves one camera and one point, so we exploit this structure using a nonlinear least-squares optimizer with an explicitly constructed sparse Jacobian pattern.

Camera rotations are parameterized using Rodrigues vectors (3 parameters) rather than full rotation matrices (9 parameters), reducing the optimization dimensionality. The first camera is fixed at the origin as a gauge constraint, preventing the solution from drifting in the global coordinate frame. We run BA every 5 frames during reconstruction and perform a final global optimization at completion.

3 Experiments and Results

3.1 Dataset Collection

We captured 332 images of two buildings' exterior walls along with surrounding bushes and trees using a smartphone camera. Images were acquired using a "crab walking" technique (moving sideways while keeping the camera facing the scene) to maximize parallax for triangulation. Corners proved especially challenging: we performed turntable-style rotations while maintaining the crab walk, capturing images at a higher frequency to ensure sufficient overlap through the turn. For bushes and vegetation, we walked around each bush in a turntable pattern to capture all viewing angles. Camera intrinsics were extracted from EXIF metadata, giving $f_x = f_y \approx 1200$ pixels with the principal point at the image center.

3.2 Validation Results

We first tested on a 30-frame subset to verify the pipeline before processing the full dataset. Two-view initialization using frames 0 and 2 (wide baseline) created 2,527 initial points.

The resulting point cloud contained 243,000 3D points covering approximately 12×10 meters. Visual inspection showed clean geometry for walls, doorways, and building features. Quantitative results showed mean reprojection error dropping from 12.3 pixels before BA to 2.1 pixels after optimization.

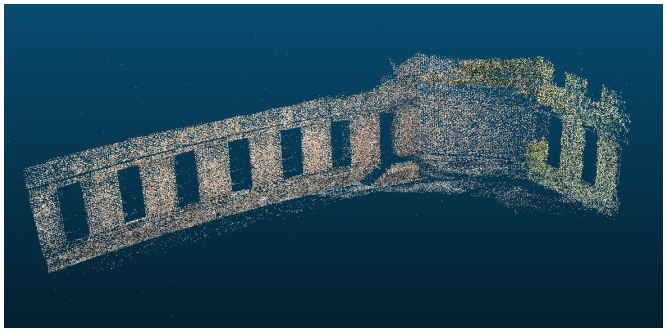

Figure 2: Reconstructed point cloud from 30-frame subset showing building walls reconstructed with minimal drift and over 200,000 points

For the full system, we processed all 332 frames using Agisoft Metashape, producing a dense point cloud of over 200,000 points with accurate camera poses. We exported this dataset for the virtual tour.

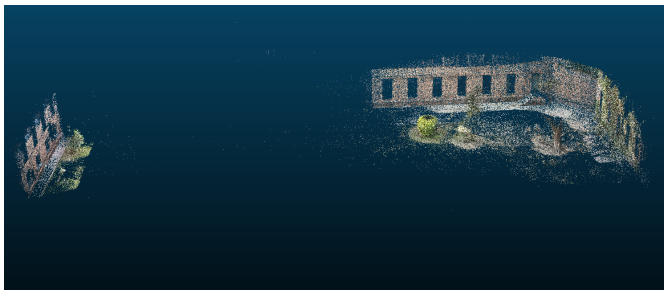

Figure 3: Point cloud reconstruction from Agisoft Metashape showing detailed geometry of building facades and surrounding vegetation

3.3 Computational Efficiency

The sliding window matcher reduced per-frame matching from over 20 minutes (exhaustive search across 100 frames) to approximately 3-4 minutes. Sparse BA with local windowing decreased optimization time from 100+ minutes for global problems to around 40 minutes for active windows of 15 cameras.

4 Discussion

4.1 Matching Strategy Analysis

The most significant computational bottleneck in SfM is feature matching. With 70,000 features per frame and many frames, exhaustive all-pairs matching would require billions of descriptor comparisons. For sequential data, this is wasteful since distant frames share minimal visual overlap.

Our hybrid strategy addresses this by matching each new frame against only 8 previous cameras: the 5 most recent (for temporal continuity) plus 3 randomly sampled older ones (for global awareness). This reduces the number of matching operations dramatically while maintaining reconstruction quality.

4.2 Tracking Failure and Recovery

Fast camera rotations (e.g., turns at room corners) caused temporary tracking loss due to motion blur and limited overlap. When matching fails to find sufficient correspondences with recent frames, the system could potentially match against visually similar but spatially distant frames, causing trajectory discontinuities.

Our hybrid matching strategy (5 recent + 3 random older cameras) addresses this by maintaining both temporal continuity and global awareness. The recent cameras provide strong matches for smooth motion, while the random older cameras can help recover if the reconstruction drifts. If PnP fails (fewer than 6 correspondences), the frame is simply skipped rather than forcing a poor registration that would corrupt the map.

Bundle Adjustment, running every 5 frames, corrects accumulated drift by jointly optimizing all poses and points. The soft L1 loss function used in BA naturally downweights outlier observations, providing robustness to occasional mismatches without requiring explicit outlier rejection in the optimization.

4.3 Geometric Instability and Outlier Removal

Initial reconstructions exhibited severe artifacts: a shapeless "mound of dirt" point cloud with points scattered far from the actual scene geometry. These issues originated from two main sources.

Small baseline triangulation: Consecutive video frames provide minimal parallax. With near-parallel viewing rays, small pixel noise amplifies into massive depth errors. For example, rays differing by 0.5° intersecting 10 meters away yield 9 cm uncertainty, but at 100 meters this becomes 87 cm. We mitigate this by using a wide baseline for initialization (frames 0 and 2) to establish robust initial scale.

Outlier pruning: To remove spurious 3D points that arise from matching errors or degenerate triangulation geometry, we implement a statistical outlier removal step. After computing the median distance from all points to the geometric center, we reject any point lying more than $4\times$ the median distance from the center. This effectively removes "exploded" points while preserving the core reconstruction. The pruning step runs before each Bundle Adjustment pass, and the correspondence maps are rebuilt to maintain index consistency after point removal.

4.4 Bundle Adjustment Scalability

Global BA on many cameras and tens of thousands of points can construct dense Jacobian matrices exceeding available RAM, causing kernel crashes. The fundamental issue is that scipy's default dense solver allocates $O(M^2)$ memory for M parameters, despite $< 1\%$ of entries being non-zero.

Our sparse formulation exploits the fact that each observation only depends on one camera and one 3D point, producing a Jacobian with mostly zero entries. We construct a sparsity pattern indicating which measurements affect which parameters (6 per camera, 3 per point). Storing only the non-zero entries reduces memory usage from gigabytes to megabytes.

We use the Trust Region Reflective (TRF) method with Jacobian scaling (`x_scale='jac'`) for automatic parameter normalization, preventing poorly-scaled problems where camera parameters and 3D coordinates differ by orders of magnitude. The `ftol=1e-3` convergence tolerance balances accuracy against runtime.

4.5 Reconstruction State Management

We encapsulate the reconstruction state in a dedicated `ReconstructionMap` class that maintains: (1) the list of 3D points and their colors, (2) camera poses indexed by frame number, (3) keypoints and descriptors for each registered camera, and (4) a correspondence dictionary mapping (camera, keypoint index) pairs to 3D point indices. This design enables efficient lookup of which keypoints in each image correspond to which 3D points, which is essential for both PnP (finding 2D-3D matches) and triangulation (avoiding duplicate point creation).

5 Virtual Tour Implementation

Figure 4 illustrates the architecture of our Three.js-based virtual tour viewer, showing the data flow from SfM outputs through scene construction to user interaction handling.

5.1 View Graph Construction

The view graph defines connectivity between camera viewpoints, determining which transitions are available to users. We construct the graph using temporal adjacency: each camera C_i is connected to its immediate neighbors C_{i-1} and C_{i+1} in the capture sequence. This sequential connectivity reflects the physical camera trajectory and ensures smooth transitions

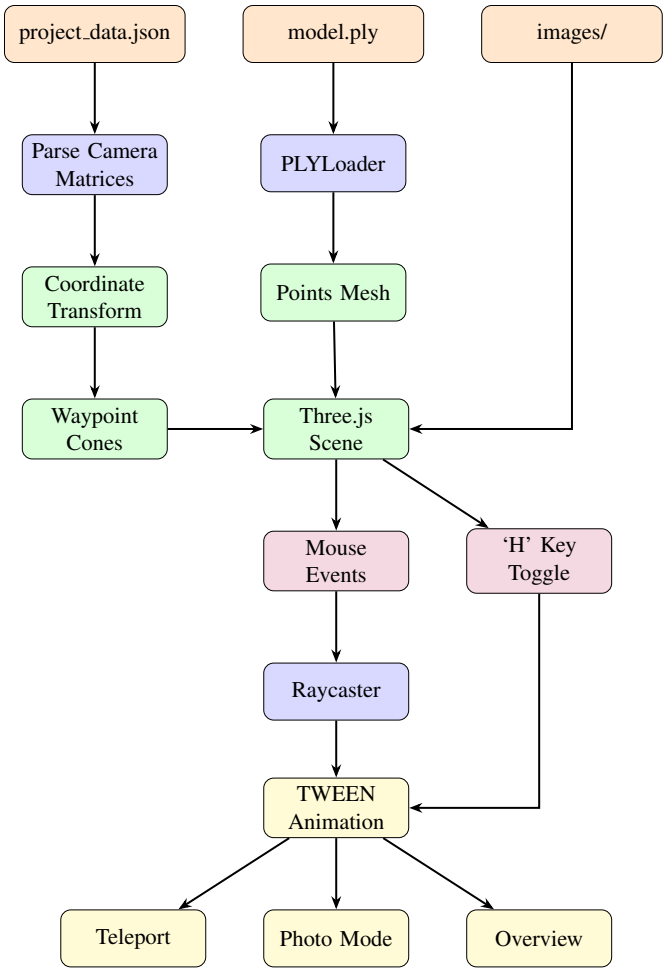

Figure 4: Virtual tour viewer architecture. SfM outputs (JSON camera data and PLY point cloud) are loaded and transformed to Three.js coordinates. User interactions trigger raycasting that animate the camera via TWEEN.js.

between nearby viewpoints. For the 332-camera dataset, this produces a linear graph structure that users traverse by clicking adjacent waypoints. The graph is implicitly encoded in the waypoint array ordering, with transitions permitted between any pair of waypoints—the animation system handles interpolation regardless of graph distance.

5.2 Scene Initialization and Data Loading

The virtual tour is implemented as a WebGL application using Three.js [5]. On initialization, the viewer fetches the camera matrices JSON file containing camera poses and the path to the PLY point cloud file. A PLY parser processes the ASCII point cloud and creates a points mesh with vertex colors extracted directly from the file. We configure point size to 0.15 world units for visibility without excessive overlap.

After loading, we compute the point cloud's bounding sphere to determine the model center, which serves as the default orbit target and overview position. The floor plane height is calculated from the average camera Y-coordinate minus 1.6 units (approximate eye height), enabling ground-plane teleportation even when clicking empty space.

5.3 Coordinate System Transformation

OpenCV and Three.js use incompatible coordinate conventions. OpenCV follows a right-handed system with Y pointing down and Z forward, while Three.js (OpenGL convention) has Y up and Z backward. We apply a 180° rotation about the X-axis to convert camera poses:

$$\mathbf{M} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{6}$$

Camera world matrices from SfM are computed as $\mathbf{T}_{\text{world}} = [\mathbf{R}^T | -\mathbf{R}^T\mathbf{t}]$, then transformed via $\mathbf{T}_{\text{three}} = \mathbf{M} \cdot \mathbf{T}_{\text{cv}} \cdot \mathbf{M}$. The 4×4 matrices are stored in column-major order for Three.js consumption. Point cloud coordinates undergo the same Y/Z flip during export.

5.4 Waypoint Visualization

Each camera position is visualized as a directional cone mesh indicating the viewing direction. We create a shared `ConeGeometry(0.3, 0.8, 16)` rotated $-\pi/2$ about X so the cone points along its local $-Z$ axis (forward). Each waypoint receives a cloned semi-transparent yellow material (`opacity: 0.6`) with `depthTest: false` to remain visible through the point cloud.

For certain frame ranges (e.g., 273–331) that exhibited incorrect orientation due to camera rotation during recording, we apply an additional Y-axis rotation correction of $-\pi/2 - 0.5$ radians at runtime, avoiding the need to reprocess the SfM pipeline. These varying corrections had to be manually determined through visual inspection.

5.5 Interaction Handling

The viewer implements three primary interaction modes through event listeners:

Hover highlighting: Mouse movement triggers raycasting against the waypoint array. When intersecting a cone, we change its material color to red and scale it to 1.3×, providing clear visual feedback. The cursor changes to a pointer to indicate clickability.

Click for photo mode: Single-clicking a waypoint triggers `enterPhotoMode()`, which animates the camera to the exact pose where the original photo was captured. Position is interpolated linearly while rotation uses quaternion spherical

interpolation (slerp) via TWEEN.js over 1.5 seconds with cubic easing. Upon completion, the corresponding image is displayed as a full-screen overlay at 90% opacity.

Double-click teleportation: Double-clicking performs raycasting against both the point cloud and an invisible floor plane. The first intersection triggers `walkToPoint()`, animating the camera horizontally while maintaining a fixed eye height (8.3 units above the floor plane) and to avoid sinking into the ground.

5.6 Overview Mode

The viewer supports a bird's-eye overview mode, toggled via the 'H' key. When activated, the camera animates to a position 20 units above and behind the model center over 2 seconds using cubic easing. In this mode, OrbitControls zoom is enabled, allowing users to examine the entire reconstruction from above. Pressing 'H' again exits overview mode, returning the camera to ground level at its current horizontal position while maintaining the viewing direction.

5.7 Animation and Rendering

All camera transitions use TWEEN.js for smooth interpolation. The animation loop runs at display refresh rate via `requestAnimationFrame`, updating OrbitControls damping, active tweens, and rendering the scene each frame. OrbitControls provide mouse-drag rotation with momentum (`dampingFactor: 0.05`), while the overlay image automatically fades to 30% opacity when the user begins rotating, allowing the 3D scene to remain visible.

6 Conclusion

This project demonstrates a complete SfM pipeline from image capture through interactive visualization. Key lessons learnt include the importance of temporal locality for computational efficiency, the necessity of multi-stage validation (two-view, incremental, global BA), and the value of adaptive thresholds for robust tracking.

The evolution from naive global matching to a hybrid temporal-random strategy with sparse optimization and statistical outlier removal represents the practical engineering required to transition theoretical algorithms into functional systems. Our final reconstruction of 300,000+ points from 332 frames validates the approach, while the Three.js virtual tour demonstrates practical applications of SfM beyond static point cloud visualization.

References

[1] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.

[2] J. L. Schonberger and J.-M. Frahm, "Structure-from-motion revisited," in *CVPR*, 2016.

[3] B. Triggs et al., "Bundle adjustment — a modern synthesis," in *Vision Algorithms: Theory and Practice*, pp. 298–372, 2000.

[4] V. Lepetit et al., "EPnP: An accurate O(n) solution to the PnP problem," *International Journal of Computer Vision*, vol. 81, no. 2, pp. 155–166, 2009.

[5] R. Cabello et al., "Three.js," `https://threejs.org`, 2010–2024.